

Debugging Convex Hull – Incorrect Orientation Logic

1. Title

Debugging Convex Hull Using Correct Orientation Logic and Optimized Hull Checking

2. Aim

To identify and correct the logical error in the orientation function used for Convex Hull construction, analyze its effect on boundary detection, and optimize the Convex Hull checking process using the Monotone Chain algorithm.

3. Problem Statement

The Convex Hull of a set of points is the smallest convex polygon that contains all the given points.

The orientation function determines whether three points form:

- A **clockwise turn**
- A **counter-clockwise turn**
- A **straight/collinear line**

The given orientation function contains an error in the calculation of the vectors, which can lead to incorrect determination of the turn direction and consequently an incorrect Convex Hull.

Given incorrect function

def orientation(p, q, r):

```
    return (q[0]-p[0])*(r[1]-q[1]) - \
           (q[1]-p[1])*(r[0]-q[0])
```

4. Logical Error

The error occurs because the second vector is calculated using:

$R - Q$

instead of:

$R - P$

For orientation, we need to calculate the cross product of:

$PQ = Q - P$

$PR = R - P$

Therefore, the correct formula is:

```
def orientation(p, q, r):
    return ((q[0] - p[0]) * (r[1] - p[1]) -
            (q[1] - p[1]) * (r[0] - p[0]))
```

5. Orientation Logic

The result of the orientation function is interpreted as follows:

Result Meaning

> 0 Counter-clockwise

< 0 Clockwise

= 0 Collinear

For example, consider:

P = (0,0)

Q = (2,0)

R = (2,2)

The calculation is:

$$(2-0)(2-0) - (0-0)(2-0)$$

$$= 4$$

Since the result is positive, the points form a **counter-clockwise turn**.

6. Impact of the Error

The orientation calculation is the core operation used to determine whether a point belongs to the boundary of the Convex Hull.

Because the original formula calculates the second vector incorrectly:

- Clockwise and counter-clockwise turns can be detected incorrectly.
- Valid boundary points may be removed.
- Some unwanted points may remain in the hull.
- The resulting polygon may not represent the actual Convex Hull.
- Boundary detection becomes unreliable.

Therefore, correcting the orientation function is essential for obtaining the correct hull.

7. Proposed Solution

The problem can be solved using **Andrew's Monotone Chain algorithm**.

The algorithm works in three major stages:

Input Points



Remove Duplicate Points



Sort Points



Build Lower Hull



Build Upper Hull



Combine Both



Final Convex Hull

8. Algorithm

Step 1

Take a collection of two-dimensional points.

Step 2

Remove duplicate points and sort the points lexicographically.

Step 3

Construct the lower hull.

For every point, check the orientation of the last two points in the hull and the current point.

If the orientation is not counter-clockwise, remove the middle point.

Step 4

Construct the upper hull using the points in reverse order.

Step 5

Combine the lower and upper hulls.

Step 6

Return the resulting points as the Convex Hull.

9. Implementation

```
def orientation(p, q, r):  
    return ((q[0] - p[0]) * (r[1] - p[1]) -  
            (q[1] - p[1]) * (r[0] - p[0]))  
  
def convex_hull(points):  
  
    # Remove duplicates and sort points  
    points = sorted(set(points))  
  
    if len(points) <= 2:  
        return points  
  
    # Lower Hull  
    lower = []  
  
    for p in points:  
        while len(lower) >= 2 and orientation(  
            lower[-2], lower[-1], p) <= 0:  
            lower.pop()  
  
        lower.append(p)
```

```
# Upper Hull
```

```
upper = []
```

```
for p in reversed(points):
```

```
    while len(upper) >= 2 and orientation(
```

```
        upper[-2], upper[-1], p) <= 0:
```

```
        upper.pop()
```

```
    upper.append(p)
```

```
# Combine lower and upper hull
```

```
return lower[:-1] + upper[:-1]
```

```
# Input
```

```
points = [
```

```
    (0, 0),
```

```
    (2, 0),
```

```
    (2, 2),
```

```
    (0, 2),
```

```
    (1, 1),
```

```
    (1, 0),
```

```
    (0, 1)
```

```
]
```

```
print("Input Points:")
```

```
print(points)
```

```
hull = convex_hull(points)
```

```
print("\nConvex Hull:")
```

```
print(hull)
```

10. Sample Input

(0,0)

(2,0)

(2,2)

(0,2)

(1,1)

(1,0)

(0,1)

11. Sample Output

Input Points:

[(0, 0), (2, 0), (2, 2), (0, 2),

(1, 1), (1, 0), (0, 1)]

Convex Hull:

[(0, 0), (2, 0), (2, 2), (0, 2)]

The points (1,1), (1,0) and (0,1) are inside the boundary, so they are not included in the final Convex Hull.

12. Optimization

A straightforward Convex Hull implementation can repeatedly check points and may take $O(n^2)$ time in some approaches.

The Monotone Chain algorithm improves the process by first sorting the points and then constructing the hull using a single scan.

Complexity

Operation	Complexity
-----------	------------

Removing duplicates	$O(n)$ average
---------------------	----------------

Sorting points	$O(n \log n)$
----------------	---------------

Lower hull	$O(n)$
------------	--------

Upper hull	$O(n)$
------------	--------

Combining hulls	$O(n)$
-----------------	--------

Overall	$O(n \log n)$
----------------	---------------------------------

Space Complexity

The algorithm requires storage for the sorted points and the two hulls.

Therefore:

Space Complexity = $O(n)$

13. Advantages

- Correctly determines point orientation.
- Efficient for large sets of points.
- Removes unnecessary interior points.
- Handles duplicate points.
- Simple and easy to implement.
- Provides an optimal $O(n \log n)$ solution for the comparison-based approach.

14. Applications

Convex Hull algorithms are used in many real-world applications:

- **Computer Graphics** – object boundaries and shape processing.
- **Geographical Information Systems (GIS)** – determining boundaries of geographical data.
- **Robotics** – obstacle and workspace boundary detection.
- **Image Processing** – detecting object shapes.
- **Collision Detection** – determining boundaries of objects.
- **Pattern Recognition** – analyzing spatial distributions.

- **Computational Geometry** – solving geometric problems efficiently.

15. Result

The logical error in the original orientation function was identified and corrected by calculating the cross product using vectors **PQ** and **PR** from the same starting point P.

The optimized Monotone Chain algorithm successfully constructs the Convex Hull with a time complexity of **$O(n \log n)$** and space complexity of **$O(n)$** .

16. Conclusion

The orientation function plays a critical role in Convex Hull construction because it determines the direction of turns between three points. The original function incorrectly used (R-Q) instead of (R-P), resulting in incorrect boundary detection.

After correcting the orientation formula and using the Monotone Chain algorithm, the Convex Hull can be generated accurately and efficiently. This approach is suitable for processing large collections of two-dimensional points.